QUANT SINGULARITY

Financial ML - Intern Screening Project

Name - Anwoy Datta
Contacts - 8005259519 / anwoy.datta@gmail.com
Github link - https://github.com/anwoy71/QS-nifty50-prediction

1. Problem Framing

Objective

The objective of this project was to build a binary classification model for predicting the next-day direction of the NIFTY 50 index. The model predicts whether the NIFTY 50 closing price on trading day T+1 will be higher or lower than the closing price on day T. The prediction horizon was restricted to one trading day ahead in order to keep the setup realistic and aligned with short-term market prediction.

Trading setup

The strategy was intentionally kept simple. Only two actions were allowed- BUY and HOLD. Sell-side and short-selling logic were not considered in this project. After the market closes on day T, the model generates a prediction for day T+1. If the prediction is positive, the strategy enters a BUY position for the next trading day. Otherwise, it remains in HOLD/cash.

An important assumption in this backtest is that positions are treated independently each day. The strategy does not model multi-day holding periods. Therefore, when the next-day prediction becomes 0, the model assumes the previous position has already been exited.

Target definition

The target variable was defined as:
1, if next-day return > 0 and 0 otherwise

Near-zero returns are often treated as market noise (usually removed using an epsilon filter). While removing very small returns could reduce noise, it also significantly reduced the already limited dataset size (around 960 examples after final cleaning and feature alignment). Since transaction-cost aware filtering was handled later during thresholding and backtest evaluation, I chose to preserve the complete directional dataset during model training.

Baseline models

Two baseline classifiers were evaluated before trusting the ML model:

- Majority-class baseline - 51.2% accuracy
- Persistence baseline (T+1 direction same as T) -58.8% accuracy

If the model cannot beat the majority baseline, it means the model is almost as good as blindly guessing the most common market direction every day.
If it cannot beat the persistence baseline, it means a simple “market continues like previous day” rule works better than the ML model.

2. Data Audit

The provided dataset consisted of daily OHLCV data for NIFTY 50, BANKNIFTY, and India VIX over the period January 2022 to December 2025, along with a starter feature file containing 31 candidate features.

Major issues:

1 . Zero Volume Entries -Certain rows contained zero trading volume values despite being regular trading sessions. These were treated as suspicious but retained when price fields appeared valid.
2 . Rolling Window Risks - All rolling statistics were verified to ensure strictly backward-looking computation. Centered rolling windows and future-aware transformations were avoided.
3 . Timestamp Alignment - All datasets were merged using inner joins on trading dates to prevent accidental forward-filling leakage across assets.
4 . Small Sample Size - The effective dataset size after cleaning and feature engineering remained relatively small (appr. 1000 observations), increasing the risk of overfitting and unstable Sharpe estimates.

I standardized all datasets by converting dates to datetime format, sorting chronologically, and applying dataset-specific prefixes before merging. Then aligned NIFTY 50, BANKNIFTY, India VIX, and Engineered Features into a master dataset using inner joins on trading dates to merge all statistics of the same day, and preventing accidental future outlook.

The **target** was constructed using next-day direction using shift(-1), and the final row(Dec 31, 2025) was removed because the future close(Jan 1, 2026) required for label generation was unavailable.

I verified all rolling window features to be backward-looking. I double-checked potential centre windowed features, since they might introduce a look-ahead bias. However manual inspection showed the correct number of NaN values; also all features had values till the last date. This confirms centre windowing may not have been used, instead purely backward rolling features were present.

I treated the starter feature file as an untrusted external dataset and excluded features with unclear construction logic, ambiguous timestamp computability, or suspicious predictive behavior. Made a final feature file that had only the required features, properly aligned, all NaN entries dropped. This acted as the dataset that could be fed directly to the model

Due to the relatively small effective sample size (approx 1000 observations before splitting), I prioritized leakage safety, validation, and low model complexity over aggressive optimization, since there are good chances of data being overfit.

3. Feature Engineering and EDA

Feature	Justification	Safe Timestamp
ret_1d	Immediate short-term momentum	EOD(T)
ret_overnight	Overnight/global sentiment gap	EOD(T)
ret_5d	Weekly trend continuation	EOD(T)
close_vs_ma5	Price relative to short trend	EOD(T)
momentum_5_20	Short vs medium trend strength	EOD(T)
vol_20d	Recent realized volatility regime	EOD(T)
vix_change	Change in market fear/volatility	EOD(T)
vix_ma_ratio	Current VIX vs recent regime	EOD(T)
bn_ret_1d	Banking sector leadership signal	EOD(T)
nifty_bn_spread	Relative NIFTY–BankNifty strength	EOD(T)
ret_zscore	Detects statistically unusual returns	EOD(T)
high_low_range	Intraday volatility/intensity proxy	EOD(T)

Note 1 - EOD(T) means End of Trade day-T
Note 2 - Features that have windowed calculation are all back-windowed

For this project, EDA is more about understanding the structure and reliability of the financial time-series data before modeling.

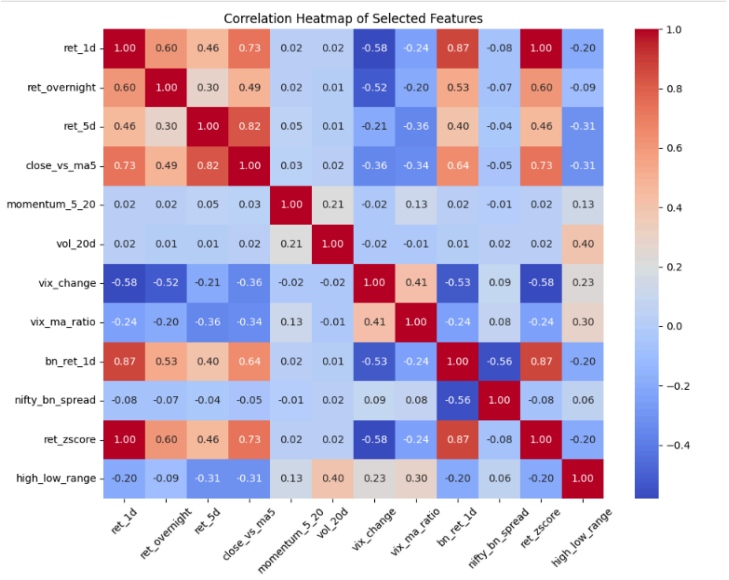

Fig - Correlation Heatmap of Selected Features

Tested several feature combinations to improve robustness while minimizing overfitting risk.

In traditional EDA practices, too much correlation can cause redundancy.

However, a particular case, where feature ‘return z-score’ had a noticable positive impact on the model performance. This means it has a very strong predictive signal, in spite of being highly correlated.

While selecting features, I did not focus only on model performance, but also on how many usable rows each feature would leave after handling NaN values. Since the dataset itself was fairly small, features with very large rolling-window NaN regions were avoided because they would significantly reduce the effective training data. I tried to maintain a balance between momentum, volatility, regime, and cross-market signals while keeping the dataset reasonably clean and stable. This trade-off helped preserve more observations for walk-forward validation while still giving good predictive performance.

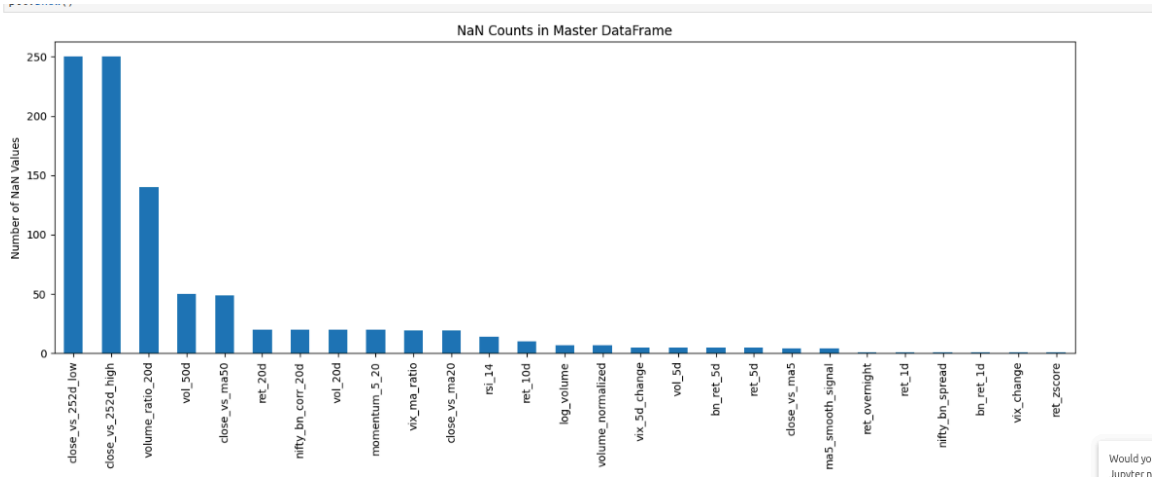

Fig - NaN Counts in Master DataFrame

4. Model and Training

Model Selection

I used an XGBoost classifier because the dataset consisted of engineered numerical time-series features with potentially non-linear relationships. Multiple model configurations and feature combinations were tested across several runs before selecting the final setup based on out-of-sample behaviour rather than only training accuracy.

Time-Series Validation Strategy

To avoid look-ahead bias, I used chronological splitting instead of random train-test splitting. Data before July 2025 was used for training, while later observations were reserved for out-of-sample testing. Within the training set, I implemented walk-forward validation using TimeSeriesSplit with 5 expanding folds so that each validation stage only used past data to predict future data.

Hyperparameter and Overfitting Control

The final XGBoost configuration was intentionally kept conservative due to the relatively small dataset size. Through repeated experiments, I observed that more complex trees improved training accuracy but reduced out-of-sample stability. Based on these observations, I selected shallow trees, a low learning rate, subsampling, and stronger regularization to reduce overfitting and improve generalization.

Model Comparison and Refinement

Different runs were compared using cross-validation accuracy, out-of-sample accuracy, balanced accuracy, confusion matrices, and trading performance metrics. During later iterations, I identified bullish prediction bias in the model outputs and experimented with probability threshold adjustments to reduce directional imbalance and improve robustness.

5. Results

The final model achieved moderate but consistent out-of-sample predictive performance across walk-forward folds. Several model configurations were evaluated, the most reliable(and not the best) metrics are as below.

Baseline comparison

- Majority-class baseline accuracy: 51.2%
- Persistence baseline accuracy: 58.8%
- Final OOS model accuracy: 54.4%
- The model outperformed the majority baseline but still remained below the persistence baseline, indicating moderate directional predictive power.

Cross-validation performance

- 5-fold walk-forward validation was used to preserve temporal ordering and avoid leakage.
- Fold accuracies ranged from 46.76% to 62.59%.
- Mean CV accuracy: 53.24%
- Validation performance varied across folds, suggesting changing market behavior across different time periods.

Out-of-sample classification metrics

- OOS Accuracy: 54.4%
  Balanced Accuracy: 53.36%
  AUC: 0.5336
- Confusion Matrix:
  True Negatives: 6
  False Positives: 55
  False Negatives: 2
  True Positives: 62
- The model still showed a bullish prediction bias, predicting upward movement more frequently than downward movement. This is the primary shortcoming of this model.

Probability threshold observations

- Threshold analysis showed that higher-confidence predictions generally improved accuracy. Accuracy improved gradually at higher probability thresholds, reaching above 63% for thresholds above 0.60.
- This suggested improved probability calibration and more meaningful confidence estimates compared to earlier model versions
- However, very high thresholds resulted in a small number of trades, so threshold tuning was kept conservative to avoid overfitting risk. The final prediction threshold was kept at 0.50.

Trading backtest performance

- Strategy returns outperformed market returns during the OOS period, even after including assumed transaction costs.
- Transaction cost assumed: 0.1% per position change.

- Sharpe Ratio (after costs): 0.9198
- Hit Rate: 50.4%
- Turnover: 16 trades
- Max Drawdown: -0.04%
- The strategy maintained relatively stable equity growth with controlled drawdowns and moderate trading frequency.

6. How Do We Know This Edge Is Real?

Statistical significance of the out-of-sample edge

I did observe that the final model slightly improved over the majority-class baseline, which initially suggested that the model had learned some directional signal. However, the improvement was still small and remained below the persistence baseline, which made me question how strong the edge really was. The OOS sample itself was only around 125 days, so even a few correct predictions could noticeably change the final accuracy and Sharpe ratio. I also noticed that higher-confidence predictions did not always become more accurate, which reduced confidence in the probability estimates. Because of this, I treated the results as suggestive rather than statistically strong. At this stage, I believe the model may contain a weak edge, but not enough evidence yet to claim robustness confidently.

Sensitivity to train/test split boundaries

I noticed that the model performance changed meaningfully across walk-forward folds. Some folds produced accuracies above 60%, while others dropped below 50%, even though the same model structure was used. This suggested that the strategy was highly dependent on the market regime and exact train-test boundary. Small changes in split dates sometimes changed OOS accuracy noticeably, which made me careful about over-interpreting a single result. I used walk-forward validation specifically to reduce leakage and mimic real deployment conditions, but the variability in results itself became an important finding. It indicated that the model was not consistently stable across all time periods.

Random-label and baseline comparison

I compared the model against simple baselines before trusting the ML results. The majority baseline represented blindly predicting the most common class(i.e up/1), while the persistence baseline assumed tomorrow behaves like today. The model managed to outperform the majority baseline, but struggled to beat persistence consistently. That was important because financial time series often contain short-term correlation, and a weak ML model can easily look good without adding real value. I did not perform a fully randomized-label experiment, but the baseline comparison itself already showed that the edge

was limited. This forced me to question whether the model was truly learning structure or partially benefiting from market direction, after numerous runs.

Ways the backtest may still be misleading

The backtest results looked stronger after threshold filtering and transaction-cost adjustment, but I realized that this could still be misleading. Increasing the probability threshold(confidence) reduced the number of trades and filtered weaker signals, which naturally improved Sharpe ratio and drawdown. However, the sample size became very small at higher thresholds. This may make metrics unstable and easier to inflate by luck (which happened during initial runs). I also noticed that the OOS period(Jun-Dec 25) itself had a generally upward market bias. Thus, bullish predictions benefited more easily. Even though I avoided look-ahead bias carefully, the dataset itself was only around 960 rows after cleaning and feature alignment, which is relatively small for financial ML. Because of this, I treated the backtest as an approximate indication rather than a robust direction.

External observations

The timeline of the data includes post covid recovery and stability period, strong bull-run in 2023 and high volatility around May-Jun 24 (General elections) and a bearish/stagnant phase, followed by recovery during 2025. The bull phases have been way intense with high volatility as compared to the bearish phase. Due to this, I believe that the model is having(or suffering) from a bullish bias.

Possible improvements

If given another week, I would first test robustness rather than trying to optimize accuracy further. I would perform rolling retraining with multiple OOS windows to check whether the edge survives across different market conditions.

An important improvement would be probability calibration, because the current confidence scores were not behaving reliably at higher thresholds. In live trading, I think the first thing that could break this approach is regime change especially if volatility structure or market direction shifts sharply.

Another interesting observation would be to isolate different segments of trade activity, specially those that have behaved abnormally. External data would be required, including news-bits, ESG and corporate actions data, etc.